



Axle Tech Exercise: Engineering

Thanks for your interest in working together. This is meant to be a fun, short exercise which will help you understand what kind of things we might work on together, and to give us a sense of your work. Please let us know if you have feedback - we're always looking to improve our process.

The exercise is in two parts:

1. **Take home component:** A short piece of work, resulting in some code that you send us
2. **Pair programming session:** we will go over your submission together and look for ways to improve it *how will pair programming work?*



We prefer Python solutions, but may be able to review solutions in other languages if you have a strong preference. Please let us know if you plan to use a different language and we'll try to accommodate it. We'll need to run your solution, so please make it clear how to do so in a [README](#).

The challenge

In this task, we're building a whitelabel app which our customers can put in the hands of their own end users to manage their electricity usage.

Your challenge is to implement a single screen for this app, namely the **charge control panel**: a screen the user can use to see what charging we have planned for their electric vehicle, and override it if necessary.

This panel is a little complex, because it needs to:

1. Show the user when the charge is scheduled to happen

2. Allow them to override the schedule and charge immediately if they need to (e.g. they get home and want to go out again in an hour, so they need to charge now)

3. Allow them to stop charging

4. Keep track of whether the current charge is the scheduled one, or the result of an override, in order to determine the behaviour of the stop charging button (see AC's below)

implement a slightly different solution for this?

user should be able to see

current battery %

if charge is scheduled charge or override charge

if scheduled charge, what will battery % be at the end of the charge

Acceptance criteria

- User should be able to see their battery % (State of Charge or SOC), what the schedule is, and what their battery SoC will be at the end of the scheduled charge.

- this obviously depends upon the starting battery %, which in reality we might get from the car's API; you can hardcode it to something like 60%

- Users should be able to hit a button to override the schedule and charge the car immediately

- When they do so, the car should charge for a set period of time (e.g. 60 minutes)

- At the end of that time, the car should revert to the schedule

- During this time, if the user hits "Stop Charge", the car should revert to the schedule

- When scheduled charging is occurring, if the user hits "Stop Charge", the schedule should be disabled until the next morning

- The charge schedule should only be applied when the user's car is plugged in - you should include a UI element that highlights whether the car plugged in or not, and makes it clear how this relates to the charging (e.g. if you hit "Start Charge" and you're not plugged in, you're not overriding the schedule: the schedule only applies when the car is plugged in)

So set current battery charge to 60% to start. Possible to get some nice of speed up button to show "fast" charging?

I've changed the "stop charge" could imply to the user that all charging is stopping when it's not

make the button more obvious what is happening

have a different screen grey out button and have an alert somewhere saying not plugged in

Template Streamlit app

- We've provided a template app implemented in Streamlit. You can use this as the basis of your submission and just implement the missing functionality.
 - The template app contains a minimal frontend that displays the relevant information.
 - There are `TODO`s in the code that indicate where you need to implement the backend functionality.
- You're also very welcome to implement your own app from scratch. This can be in a server-rendered app, or have separate back and frontends.

What we'd like you to submit

- Some runnable code, packaged however you want
 - You don't need to worry about real price data or the actual optimization - please just mock this with something reasonable (e.g. "charge from 2-5am")
 - Obviously we're not expecting this to connect to a real car - you should be mocking the car with an object that behaves a bit like a car 
 - We don't mind whether the UI is optimized for desktop or mobile, but it should be intuitive and attractive for end users 
- Brief notes on the decisions you made as you went through the task; these could also be as comments in the code
 - what assumptions did you make?
 - which parts did you choose to spend time on; which bits did you consider less important?
 - how you designed the system bearing in mind its end use.

Extra things to think about:

- Testing
- Performance
- Deployment

• Testing

• Performance

• Deployment

- Set up view to run locally

• Open Q's

- what are the format json scripts doing in there? I just remember those

• Assumptions

- It seems like the whole script gets run - run every time there's an external state change. Is this SteamKit best practice or is this a better way of doing this?

• Prototyping time

- Decided not to start from scratch with a new web app since that's what this took in about.
- getting debug mode setup
- reading through setup in debug and understanding SteamKit basics before starting
- Putting in my own to do's that correspond to tools

• End user

consideration

- changed SOC to just battery %
↳ who uses SOC?!

• Improvements

- Change button colours for on/off

Notes

- each time a button is pushed the whole thing runs again
- make sure all TODOS are removed

	A	B	C	D	Not plugged in
Scheduled charging					
Override					

← shouldn't be possible

	Control 1	Control 2
A	_____	stop override
B	stop ^{scheduled} charging until specified time	start override
C	_____	_____
D	start ^{scheduled} charging	_____
NPI	_____	_____

Adrian user to set desired percentage